



# 3.Hive数据类型详解

## 3.1 基本数据类型(☆☆理解)

### 3.1.1 基本数据类型详解 (☆☆理解)

Hive中的数据类型指的是表中列的字段类型，主要分为两类：基础数据类型、集合数据类型。

【备注：数据类型英文字母大小写不敏感】

| Hive      | MySQL     | Java    | 长度                                        | 例子                                      |
|-----------|-----------|---------|-------------------------------------------|-----------------------------------------|
| tinyint   | tinyint   | byte    | 1byte 有符号整数                               | 2                                       |
| smallint  | smallint  | short   | 2byte 有符号整数                               | 20                                      |
| int       | int       | int     | 4byte 有符号整数                               | 2000                                    |
| bigint    | bigint    | long    | 8byte 有符号整数                               | 20000000                                |
| boolean   | 无         | boolean | 布尔类型，true 或者 false                        | true false                              |
| decimal   | decimal   | decimal | 可带小数的精确数字字符串，常用于交易金额的计算，从 Hive0.11.0 开始支持 | 3.22                                    |
| float     | float     | float   | 单精度浮点数                                    | 3.14159                                 |
| double    | double    | double  | 双精度浮点数                                    | 3.14159                                 |
| string    | varchar   | string  | 字符系列。可以指定字符集。可以使用单引号或者双引号。                | 'now is the time'<br>"for all good men" |
| timestamp | timestamp |         | 时间类型                                      |                                         |
| binary    | binary    |         | 字节数组                                      |                                         |
| varchar   | varchar   | varchar | 从 Hive0.11.0 开始支持，长度不定字符串；字符数范围 1 - 65535 |                                         |
| char      | char      | char    | 从 Hive0.11.0 开始支持，长度固定字符串；最大的字符数：255      |                                         |
| date      | date      | date    | 从 Hive0.11.0 开始支持，日期，内容格式：YYYY-MM-DD      | 2023-11-11                              |

## 1. 数字类型

### 整数类型

Hive有4种带符号的整数类型：TINYINT, SMALLINT, INT, BIGINT，分别对应Java中的byte, short, int, long。字节长度分别为1, 2, 4, 8字节。在使用整数字面量时，默认情况下为INT，如果要声明为其他类型，通过后缀来标识。

### **浮点数类型：**

浮点数默认会当作DOUBLE型

单精度，也就是 float，在 32 位机器上用 4 个字节来存储的；而双精度double是用 8 个字节来存储的，这是它们最本质的区别。由于存储位不同，它们能表示的数值的范围就不同，也就是能准确表示的数的位数就不同。

- float的精度是6位有效数字，取值范围是10的-38次方到10的38次方，float占用4字节空间。
- double的精度是15位有效数字，取值范围是10的-308次方到10的308次方，double占用8字节空间。
- DECIMAL(10,2)代表最多10位数字，后2位是小数。此时也就是说，小数点前最多有10位数字，如果超过一位则会变成null。
- 如果不指定参数，那么默认是DECIMAL(10,0)，即没有小数位，此时0.82会变成1。

## **2. 字符串类型**

### **关于varchar与char的说明：**

(1) varchar属于可变长的字符类型，范围1-65535；而char属于固定长度的字符类型，最大长度255。

(2) 假定声明了varchar(20)与char(20)两种数据类型，当存入的字符占用小于20时，声明为varchar的字符只占用足够表示它的那些字符空间；而char则仍然占满20个字节空间，用空格填充。

### **关于string的说明：**

对于Hive的string类型相当于数据库的varchar类型，该类型是一个可变的字符串，不过它不能声明其中最多能存储多少个字符，理论上它可以存储2GB的字符数。可以用单引号 (') 或双引号 (") 定义，这个类型是以后我们定义字符串的常用类型。

### **char和varchar区别对比：**

建表：

```
create table ds_hive.ch3_chr_str_test(chr1 char(10), chr2 char(20), str1 varchar(10), str2 varchar(10))
```

插入数据:

```
insert into ds_hive.ch3_chr_str_test values('aa','aa ','bb','bb ')
```

对比区别:

```
select chr1=chr2, str1=str2 from ds_hive.ch3_chr_str_test;
```



```
Time taken: 0.427 seconds, Fetched: 2 row(s)
hive (default)> select chr1=chr2, str1=str2 from ds_hive.ch3_chr_str_test;
Query ID = hewwen8888_20250531140549_1902534f-e2ff-4dcb-9f13-fa664971ef58
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1716866155638_240595, Tracking URL = http://ds-bigdata-001:8088/proxy/application_1716866155638_240595/
Kill Command = /opt/cloudera/parcels/CDH/lib/hadoop/bin/hadoop job -kill job_1716866155638_240595
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2025-05-31 14:05:56,941 Stage-1 map = 0%, reduce = 0%
2025-05-31 14:06:03,116 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.19 sec
MapReduce Total cumulative CPU time: 2 seconds 190 msec
Ended Job = job_1716866155638_240595
25/05/31 14:06:05 ERROR util.JobUtil: Exception: cannot make a new request because the previous request is still in progress
MapReduce Jobs Launched:
Stage: Stage-1: Map: 1 Cumulative CPU: 2.19 sec HDFS Read: 6008 HDFS Write: 110 HDFS EC Read: 0
Total MapReduce CPU Time Spent: 2 seconds 190 msec
OK
true      false
Time taken: 16.227 seconds, Fetched: 1 row(s)
hive (default)>
```

char对比不受空格影响, varchar受空格影响

### 3. 日期与时间戳

**TIMESTAMP**: 存储纳秒级别的时间戳, 同时Hive提供了一些内置函数用于在TIMESTAMP与Unix时间戳(秒)和字符串之间做转换。

**Date**: Hive中的Date只支持YYYY-MM-DD格式的日期, 其余写法都是错误的, 如需带上时分秒, 请使用timestamp。

获取当前timestamp:

```
current_timestamp() --返回值: timestamp
```

获取当前日期:

```
current_date() --返回值: date
```

获取当前时间戳:

```
unix_timestamp()
```

获取指定字符串的Unix时间戳:

```
select unix_timestamp('2023-12-05 09:00:00');
```

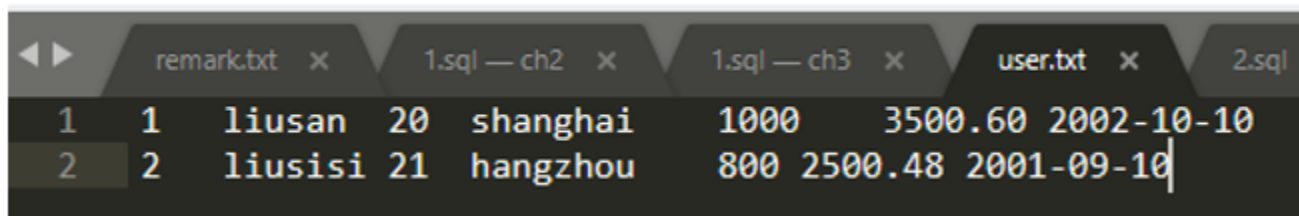
将指定时间戳转化为日期格式 (企业常用, 一般指定时间维度动态参数):

```
from_unixtime(unix_timestamp('2023-12-05 09:00:00'), 'yyyy-MM-dd')
```

### 3.1.2 基础数据类型应用示例 (☆☆理解)

这里给大家举个例子, 演示一下各个数据类型的使用。

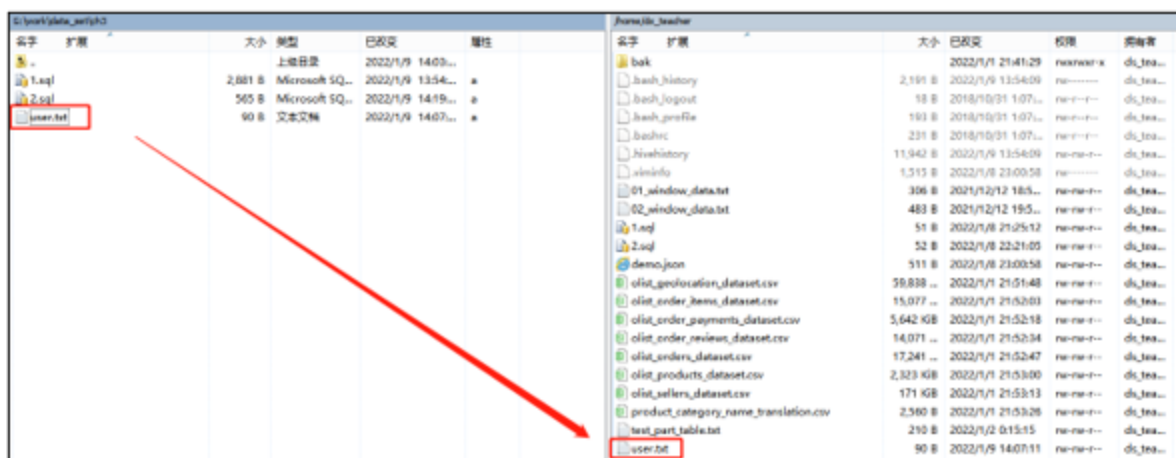
我们有个文本文件ch3\_user.txt: 每行表示一条用户信息, 每行数据中间用tab分隔开。



|   | 1 | 2       | 3  | 4        | 5    | 6                  |
|---|---|---------|----|----------|------|--------------------|
| 1 | 1 | liusan  | 20 | shanghai | 1000 | 3500.60 2002-10-10 |
| 2 | 2 | liusisi | 21 | hangzhou | 800  | 2500.48 2001-09-10 |

我们现在利用这个文件来新建对应的表, 将其存储到hive中进行分析。

(1) 首先通过winscp上传到服务器上面:



## (2) 新建表:

```
Use ds_hive;
CREATE TABLE ds_hive.ds_hive_user_info (
  user_id int comment '用户id',
  user_name string comment '用户姓名',
  user_age int comment '用户年龄',
  user_city string comment '城市名称',
  user_city_person bigint comment '城市人口',
  consume_amount decimal(20,2) comment '用户消费金额',
  user_birthday date comment '用户出生年月'
) comment '用户信息表'
row format delimited
fields terminated by '\t'
stored as textfile;
```

--思考为什么用textfile?

## (3) 载入实验数据:

---这个路径要查看你自己的文件到底在哪个目录下

```
load data local inpath '/home/hewwen8888/data/ch3_user.txt' overwrite into table ds_hive.ds_hive_u
```

查看实验数据:

```
WARNING: hive CLI is deprecated and migration to Beeline is recommended.
hive (default)> select * from ds_hive.ds_hive_user_info;
OK
1      liusan  20      shanghai      1000      3500.60  2002-10-10
2      liusisi 21      hangzhou      800      2500.48  2001-09-10
Time taken: 1.761 seconds, Fetched: 2 row(s)
hive (default)>
```

#### (4) 使用实验数据:

```
hive (default)> select '2'+user_age from ds_hive.ds_hive_user_info;
hive (ds_hive)> select cast('2' as int) +user_age from ds_hive.ds_hive_user_info;
```

```
Time taken: 1.761 seconds, Fetched: 2 row(s)
hive (default)> select cast('2' as int) +user_age from ds_hive.ds_hive_user_info;
Query ID = hewwen8888_20250531141354_46571049-1507-4a6d-a8ac-8ef670a2bbd5
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1716866155638_240599, Tracking URL = http://ds-bigdata-001:8088/proxy/application_1716866155638_240599/
Kill Command = /opt/cloudera/parcels/CDH/lib/hadoop/bin/hadoop job -kill job_1716866155638_240599
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2025-05-31 14:14:02,432 Stage-1 map = 0%, reduce = 0%
2025-05-31 14:14:08,856 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.7 sec
MapReduce Total cumulative CPU time: 2 seconds 700 msec
Ended Job = job_1716866155638_240599
25/05/31 14:14:09 ERROR util.JobUtil: Exception: cannot make a new request because the previous response is still open: please
MapReduce Job Launched:
Stage-Stage 1: Map: 1 Cumulative CPU: 2.7 sec HDFS Read: 5205 HDFS Write: 117 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 700 msec
OK
22
23
Time taken: 15.694 seconds, Fetched: 2 row(s)
hive (default)> select '2'+user_age from ds_hive.ds_hive_user_info;
Query ID = hewwen8888_20250531141427_9f0ba7f6-ea0b-4c85-82f5-7715d0ac3723
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1716866155638_240600, Tracking URL = http://ds-bigdata-001:8088/proxy/application_1716866155638_240600/
Kill Command = /opt/cloudera/parcels/CDH/lib/hadoop/bin/hadoop job -kill job_1716866155638_240600
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2025-05-31 14:14:33,706 Stage-1 map = 0%, reduce = 0%
2025-05-31 14:14:39,808 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.65 sec
MapReduce Total cumulative CPU time: 2 seconds 650 msec
Ended Job = job_1716866155638_240600
25/05/31 14:14:40 ERROR util.JobUtil: Exception: cannot make a new request because the previous response is still open: please
MapReduce Job Launched:
Stage-Stage 1: Map: 1 Cumulative CPU: 2.65 sec HDFS Read: 5393 HDFS Write: 121 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 650 msec
OK
22.0
23.0
```

转为整型

字符型参与计算会转为浮点型

## 3.2 复杂数据类型详解（集合数据类型）

Hive有三种复杂数据类型array、map和struct。array和map与Java中的array和map类似，而struct由一组称为成员的不同数据组成，其中每个成员可以具有不同的类型。

| 类型     |                                                        |
|--------|--------------------------------------------------------|
| ARRAY  | 数组，由一系列相同数据类型的元素组成，这些元素可以通过下标来访问。比如有一个ARRAY            |
| MAP    | 字典，包含key->value键值对，可以通过key来访问元素。比如"userlist"是一个map类型，其 |
| STRUCT | 结构体，可以包含不同数据类型的元素。这些元素可以通过"点语法"的方式来得到所需要的              |

## 3.2.1 复杂数据类型应用示例

(1) 创建带有复杂结构的测试表ds\_hive.ch3\_employees：

```
CREATE TABLE ds_hive.ch3_employees(
  name STRING,
  salary FLOAT,
  subordinates ARRAY<STRING>,
  deductions MAP<STRING, FLOAT>,
  address STRUCT<street:STRING, city:STRING, state:STRING, zip:INT>
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','      -- 列分隔符
COLLECTION ITEMS TERMINATED BY '_' -- STRUCT 和 ARRAY 的分隔符
MAP KEYS TERMINATED BY ':' -- MAP中的key与value的分隔符
LINES TERMINATED BY '\n' -- 行分隔符
stored as textfile;
```

查看数据，假设某表有如下一行，我们用JSON格式来表示其数据结构。在Hive下访问的格式为：

```
{
  "name": "John Doe",
  "salary": 100000.0,
  "subordinates": ["Mary Smith", "Todd Jones"], //列表Array, subordinates[1]="Todd Jones"
  "deductions": { //键值Map, deductions['Federal Taxes']=0.2
    "Federal Taxes": 0.2,
    "State Taxes": 0.05,
    "Insurance": 0.1
  },
  "address": { //结构Struct, address.city="Chicago"
    "street": "1 Michigan Ave.",
    "city": "Chicago",
    "state": "IL",
    "zip": 60600
  }
}
```

基于上述数据结构，我们在Hive里创建对应的表，并导入数据。

## (2) 创建本地测试文件ch3\_employees.txt：

```
John Doe,100000.0,Mary Smith_Todd Jones,Federal Taxes:0.2_State Taxes:0.05_Insurance:0.1,1 Michigan Ave.,Chicago,IL,60600
Tom Smith,90000.0,Jan_Hello Kitty,Federal Taxes:0.2_State Taxes:0.05_Insurance:0.1,Guang dong,China,0.5L,60661
```

注意，STRUCT和ARRAY里的元素间关系都可以用同一个字符表示，这里用“\_”。

**(3) 导入文本数据到测试表，访问三种集合列里的数据**，以下分别是ARRAY，MAP，STRUCT的访问方式。

```
load data local inpath "/home/hewwen8888/data/ch3_employees.txt" overwrite into table ds_hive.ch3_employees;
select subordinates[1], deductions['Federal Taxes'], address.city from ds_hive.ch3_employees;
```

查看导入的数据：

```
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive (default)> select* from ds_hive.ch3_employees;
OK
John Doe      100000.0      ["Mary","Smith","Todd","Jones"] {"FederalTaxes":0.2,"State Taxes":0.05,"Insurance":0.1} {"street":"1 MichiganAve.", "city":"Chicago", "state":"IL", "zip":60600}
Tom Smith     90000.0      ["Jan","Hello","Ketty"] {"FederalTaxes":0.2,"StateTaxes":0.05,"Insurance":0.1} {"street":"Guangdong.", "city":"China", "state":"0.5L", "zip":60661}
Time taken: 1.749 seconds, Fetched: 2 row(s)
hive (default)>
```

查看取数的结果：



```

hive (default)> select* from ds_hive.ch3_employees;
OK
John Doe      100000.0      [{"Mary","Smith","Todd","Jones"} {"FederalTaxes":0.2,"State Taxes":0.05,"Insurance":0.1} {"street":"1 MichiganAve.", "city"
Tom Smith     90000.0      [{"Jan","Hello","Ketty"} {"FederalTaxes":0.2,"StateTaxes":0.05,"Insurance":0.1} {"street":"Guangdong.", "city":"China", "state":"0.
Time taken: 1.749 seconds, Fetched: 2 row(s)
hive (default)> select subordinates[0] deductions['FederalTaxes'].address.city from ds_hive.ch3_employees;
Query ID = hewwen8888_20250531151435_1716866155638_240613_090d4743_9534_f109bf64098b
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1716866155638_240613, Tracking URL = http://ds-bigdata-001:8088/proxy/application_1716866155638_240613/
Kill Command = /opt/cloudera/parcels/CDH/lib/hadoop/bin/hadoop job -kill job_1716866155638_240613
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2025-05-31 15:14:41,002 Stage-1 map = 0%, reduce = 0%
2025-05-31 15:14:47,121 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.87 sec
MapReduce Total cumulative CPU time: 2 seconds 870 msec
Ended Job = job_1716866155638_240613
25/05/31 15:14:48 ERROR util.JobUtil: Exception: cannot make a new request because the previous response is still open: please call response.close()
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.87 sec HDFS Read: 5364 HDFS Write: 142 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 870 msec
OK
Mary      0.2      Chicago
Jan        0.2      China
Time taken: 12.877 seconds, Fetched: 2 row(s)
hive (default)>

```

array根据下标取  
map根据key  
struct根据关键字

## 通过集合类型来定义列的好处是什么？

在大数据系统中，不遵循标准格式的一个好处就是可以提供更高吞吐量的数据。当处理的数据的数量级是T或者P时，以最少的"头部寻址"来从磁盘上扫描数据是非常必要的。按数据集进行封装的话可以通过减少寻址次数来提高查询的速度。而如果根据外键关系关联的话则需要进行磁盘间的寻址操作，这样会有非常高的性能消耗。

## 3.3 数据类型转换

### 隐式转换

Hive在需要的时候将会对numeric类型的数据进行隐式转换。比如我们对两个不同数据类型的数字进行比较，假如一个数据类型是INT型，另一个是SMALLINT类型，那么SMALLINT类型的数据将会被隐式转换地转换为INT类型，这个到底和Java中的一样。但是我们不能隐式地将一个INT类型的数据转换成SMALLINT或TINYINT类型的数据，这将会返回错误，除非你使用了CAST操作。

#### 隐式类型转换规则如下：

- (1) 任何整数类型都可以隐式地转换为一个范围更广的类型，如tinyint可以转换成int，int可以转换成bigint。
- (2) 所有整数类型、float和string类型都可以隐式地转换成double。
- (3) tinyint、smallint、int都可以转换为float。
- (4) boolean类型不可以转换为任何其它的类型。

|           | boolean | tinyint | smallint | int   | bigint | float | double | decimal | string | varchar | timestamp | date  | binary |
|-----------|---------|---------|----------|-------|--------|-------|--------|---------|--------|---------|-----------|-------|--------|
| boolean   | true    | false   | false    | false | false  | false | false  | false   | false  | false   | false     | false | false  |
| tinyint   | false   | true    | true     | true  | true   | true  | true   | true    | true   | true    | false     | false | false  |
| smallint  | false   | false   | true     | true  | true   | true  | true   | true    | true   | true    | false     | false | false  |
| int       | false   | false   | false    | true  | true   | true  | true   | true    | true   | true    | false     | false | false  |
| bigint    | false   | false   | false    | false | true   | true  | true   | true    | true   | true    | false     | false | false  |
| float     | false   | false   | false    | false | false  | true  | true   | true    | true   | true    | false     | false | false  |
| double    | false   | false   | false    | false | false  | false | true   | true    | true   | true    | false     | false | false  |
| decimal   | false   | false   | false    | false | false  | false | false  | true    | true   | true    | false     | false | false  |
| string    | false   | false   | false    | false | false  | false | true   | true    | true   | true    | false     | false | false  |
| varchar   | false   | false   | false    | false | false  | false | true   | true    | true   | true    | false     | false | false  |
| timestamp | false   | false   | false    | false | false  | false | false  | false   | true   | true    | true      | false | false  |
| date      | false   | false   | false    | false | false  | false | false  | false   | true   | true    | false     | true  | false  |
| binary    | false   | false   | false    | false | false  | false | false  | false   | false  | false   | false     | false | true   |

## 显式转换

我们可以用CAST来显式的将一个类型的数据转换成另一个数据类型。

**语法：** `cast(expr as <type>)`

**返回值：** Expected "=" to follow "type"

**说明：** 返回转换后的数据类型

**举个例子：** 假如我们一个员工表employees，其中有name、salary等字段；salary是字符串类型的。有如下的查询：

**案例实操：**

```
hive (default)> select '1' + 2, cast('1' as int) + 2;
```

```
_c0    _c1
3.0
```

```
SELECT name, salary FROM employees
WHERE cast(salary AS FLOAT) < 100000.0;
```

这样salary将会显示的转换成float。如果salary是不能转换成float，这时候cast将会返回NULL！

对cast有以下几点需要说明的：

- 如果将浮点型的数据转换成int类型的，内部操作是通过round()或者floor()函数来实现的，而不是通过cast实现！
- 对于Date类型的数据，只能在Date、Timestamp以及String之间进行转换。

下表将进行详细的说明：

| 有效的转换                   | 结果                                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------|
| cast(date as date)      | 返回 date 类型                                                                                     |
| cast(timestamp as date) | timestamp 中的年/月/日的值是依赖于当地的时区，结果返回 date 类型                                                      |
| cast(string as date)    | 如果 string 是 YYYY-MM-DD 格式的，则相应的年/月/日的 date 类型的数据将会返回；但如果 string 不是 YYYY-MM-DD 格式的，结果则会返回 NULL。 |
| cast(date as timestamp) | 基于当地的时区，生成一个对应 date 的年/月/日的时间戳值                                                                |
| cast(date as string)    | date 所代表的年/月/日时间将会转换成 YYYY-MM-DD 的字符串。                                                         |

### 3.4 数据类型企业实战(☆☆☆实操)

#### 3.4.1 需求背景

企业一般会通过埋点来收集页面的浏览和点击数据，收集的数据会以json格式保存在文本中，这个时候我们就需要将数据清洗入库，然后分析，本次需求带领大家阿里移动电商数据集做了初步的数据分析，通过数据分析我们能对业务做出更好的洞察并进一步采取Action。

### 3.4.2 数据描述

假设我们有一张用户点击网站表，是一个json格式的文本，name是用户的姓名，age是用户的年龄，websites是用户点击的页面数据，是一个json格式的字符串，存储点击页面和时间，具体结构如下：

```
{
  "name": "zhangsan",
  "age": 30,
  "websites": [
    {"website": "baidu.com", "time": "2021-10-01"},
    {"website": "google.com", "time": "2021-10-01"},
    {"website": "google.com", "time": "2021-10-01"}
  ]
}
```

具体数据：

```
{"name": "zhangsan", "age": 30, "websites": [{"website": "baidu.com", "time": "2021-10-01"}, {"website": "google.com", "time": "2021-10-01"}],}
{"name": "lisi", "age": 28, "websites": [{"website": "baidu.com", "time": "2021-10-01"}, {"website": "google.com", "time": "2021-10-01"}],}
```

### 3.4.3 需求描述

1. 我们将文本数据落表，表结构如下
2. 并统计用户点击网站的数量。

**PS：**企业一般会用json格式将数据保存为文本格式，多用来处理用户的浏览和点击等，比如淘宝网的浏览，游戏网页的点击等场景，此类数据一般量级较大，我们一般命名为用户行为数据，保存的文本格式的数据的数据入库，并进行清洗。

### 3.4.4 需求开发

#### 1. 数据准备

```
create table ds_hive.ch3_web_list(text string);
insert into ds_hive.ch3_web_list
values('{"name": "zhangsan","age": 30,"websites": [{"website":"baidu.com","time":"2021-10-01"},{"w
insert into ds_hive.ch3_web_list
values('{"name": "lisi","age": 28,"websites": [{"website":"baidu.com","time":"2021-10-01"},{"websites": [{"website":"baidu.com","time":"2021-10-01"}]}
```

解析json，形成三个字段，需要用到两个函数get\_json\_object()或json\_tuple()这两个内置函数：

get\_json\_object()：每个字段单独调用

json\_tuple()：一次调用多个字段

```
select
  get_json_object(text, '$.name') as name,
  get_json_object(text, '$.age') as age,
  get_json_object(text, '$.websites') as websites
from ds_hive.ch3_web_list;
```

或者

```
select json_tuple(text, 'name', 'age', 'websites') as (name,age,websites)
from ds_hive.ch3_web_list;
```

## 2. 按照'|'进行切分分成两个json字符串组成的数组

需要用到regexp\_replace函数

#将字符串A中的符合java正则表达式B的部分替换为C。

```
regexp_replace(string A, string B, string C)
```

```

select
  get_json_object(text, '$.name') as name,
  get_json_object(text, '$.age') as age,
  split(
    regexp_replace(
      regexp_replace(
        get_json_object(text, '$.websites'),
        '\\[|\\]',
        ''
      ),
      '\\}\\|\\|\\{',
      '\\}\\|\\|\\{ '
    ),
    '\\| '
  ) as websites
from ds_hive.ch3_web_list;

```

这段SQL:

步骤1: 获取 websites 字段

步骤2: 去掉方括号 [和 ]

步骤3: 将 },{替换为 }{|

步骤4: 用 |分割成数组

### 3. 将数组中两个字符串变成两行数据，需要用到explode语法

explode的基础语法为: `explode(Array OR Map)`

explode()函数接收一个array或者map类型的数据作为输入，然后将array或map里面的元素按照每行的形式输出，即将hive一列中复杂的array或者map结构拆分成多行显示，也被称为列转行函数。

**方法1: 根据数组下标拿到数据,只拿该数组第一个数据[0]**

```
select
  t1.name,
  t1.age,
  split(
    regexp_replace(
      regexp_replace(websites, '\\[|\\\\', ''),
      '\\}\\}\\,\\\\{',
      '\\}\\}\\|\\\\{'
    ),
    '\\|'
  )[0] as website1
from (
  select
    get_json_object(text, '$.name') as name,
    get_json_object(text, '$.age') as age,
    get_json_object(text, '$.websites') as websites
  from ds_hive.ch3_web_list
) t1;
```

## 方法2：爆炸函数炸开数组，获取该数组的所有数据

```

SELECT
  get_json_object(text, '$.name') as name,
  get_json_object(text, '$.age') as age,
  web as json
FROM ds_hive.ch3_web_list
LATERAL VIEW explode(
  split(
    regexp_replace(
      regexp_replace(
        get_json_object(text, '$.websites'),
        '\\[|\\]',
        ''
      ),
      '\\}\\|\\|\\|\\{',
      '\\}\\|\\|\\|\\{
    ),
    '\\|'
  ) tmp_table AS web;

```

#### 4. 再用get\_json\_object进行解析其中的website和time数据

##### 方法1:



```

select
    t2.name,
    t2.age,
    get_json_object(website1,'$.website') as website,
    get_json_object(website1,'$.time') as time
from (
    select
        t1.name,
        t1.age,
        split(
            regexp_replace(
                regexp_replace(websites, '\\[|\\]', ''),
                '\\}\\}\\,\\{',
                '\\}\\}\\|\\{'
            ),
            '\\|'
        )[0] as website1
    from (
        select
            get_json_object(text,'$.name') as name,
            get_json_object(text,'$.age') as age,
            get_json_object(text,'$.websites') as websites
        from ds_hive.ch3_web_list
    ) t1
) t2;

```

**方法2:**

```

SELECT
  get_json_object(text, '$.name') as name,
  get_json_object(text, '$.age') as age,
  get_json_object(web, '$.website') as website,
  get_json_object(web, '$.time') as time
FROM ds_hive.ch3_web_list
LATERAL VIEW explode(
  split(
    regexp_replace(
      regexp_replace(
        get_json_object(text, '$.websites'),
        '\\[|\\]',
        ''
      ),
    ),
    '\\}\\}\\,\\{\\',
    '\\}\\}\\|\\{\\'
  ),
  '\\|'
) tmp_table AS web;

```

## 5. 将数据插入用户点击表

